

PART A

Algorithm-1

```
max<-arr[0]
  for i<-1 to arr.length-1 do
    if arr[i]>max
max<-arr[i]

secondMax<-arr[0]
  for j<-1 to arr.length-1 do
    if arr[j]>secondMax and arr[j]<max
secondMax<-arr[j]

thirdMax<-arr[0]
  for k<-1 to arr.length-1 do
    if arr[k]>thirdMax and arr[k]<secondMax
thirdMax<-arr[k]
```

Algorithm-2

```
prepreMax<-arr[0]
preMax<-arr[0]
max<-arr[0]
for i<-1 to arr.length-1 do
  if arr[i]>=max then
    prepreMax<-preMax
    preMax<-max
    max<-arr[i]
  else if arr[i]>=preMax then
    prepreMax<-preMax
    preMax<-arr[i]
  else if arr[i]>=prepreMax then
    prepreMax=arr[i]
```

Algorithm-3

```
for i<-0 to arr.length-1 do
  k<-arr[i]
  if k is in the map then
    map[k]<-map[k]+1
  else map[k]<-1
count<-0
for each key in the map in descending order do
  for j<-0 to map[key] do
    print key
    count<-count+1
    if count==3 then break
  if count==3 then break;
```

PART B

B) TIME COMPLEXITY ANALYSIS

1) Algorithm - 1- Three for loops

Line	primitive Operations	Count
		2
$\text{max} \leftarrow \text{arr}[0]$	1 assign + 1 index	$n+1$
for $i \leftarrow 1$ to $n-1$ do	1 assign + n Comparison	$2(n-1)$
if $\text{arr}[i] > \text{max}$ then	1 index + 1 compare	$2(n-1)$
$\text{max} \leftarrow \text{arr}[i]$	1 assign + 1 index	2
$\text{SecondMax} \leftarrow \text{arr}[0]$	1 assign + 1 index	$n+1$
for $j \leftarrow 1$ to $n-1$ do	1 assign + n Comparisons	$4(n-1)$
if $\text{arr}[j] > \text{SecondMax}$	2 index + 2 compare	$2(n-1)$
and $\text{arr}[j] < \text{max}$	1 assign + 1 index	2
$\text{SecondMax} \leftarrow \text{arr}[j]$	1 assign + 1 index	$1+n$
$\text{thirdMax} \leftarrow \text{arr}[0]$	1 assign + n Comparisons	$4(n-1)$
for $k \leftarrow 1$ to $n-1$ do	2 index + 2 compare	$2(n-1)$
if $\text{arr}[k] > \text{thirdMax}$	1 assign + 1 index	
and $\text{arr}[k] < \text{SecondMax}$		
$\text{thirdMax} \leftarrow \text{arr}[k]$		

$$\begin{aligned} \hookrightarrow \text{Total Count}(T(n)) &= 6 + 3(1+n) + 4(2)(n-1) + 2(4)(n-1) \\ &= 23n - 14 \end{aligned}$$

$$\begin{aligned} \text{We skip the constant} &= \cancel{23n} - \cancel{14} = n \\ &= \underline{\underline{O(n)}} \end{aligned}$$

↳ Algorithm-2 One loop (if/else)

Line	primitive Operations	Count
prepreMax $\leftarrow$ arr[0]	1 assign + 1 index	2
preMax $\leftarrow$ arr[0]	1 assign + 1 index	2
max $\leftarrow$ arr[0]	1 assign + 1 index	2
for i $\leftarrow$ 1 to n-1 do	1 assign + n Comparisons	1+n
if arr[i] $\geq$ max then	1 index + 1 compare	2(n-1)
prepreMax $\leftarrow$ preMax	1 assign	n-1
preMax $\leftarrow$ max	1 assign	n-1
max $\leftarrow$ arr[i]	1 assign + 1 index	2(n-1)
else if arr[i] $\geq$ prepreMax then	1 index + 1 compare	n-1
prepreMax $\leftarrow$ preMax	1 assign	n-1
preMax $\leftarrow$ arr[i]	1 assign + 1 index	n-1
else if arr[i] $\geq$ prepreMax then	1 index + 1 compare	n-1
prepreMax $\leftarrow$ arr[i]	1 assign + 1 index	2(n-1)

$$\text{Total Count (TC)} = 6 + (1+n) + 14(n-1)$$

$$= 15n - 7$$

We skip the constants

O(n)

$$15n - 7 = n$$

↳ Algorithm 3 - Ordered Dictionary (TreeMap)

Line	primitive Operations	Count
for $i \leftarrow 0$ to $n-1$ do	1 assign + n comparisons	$1+n$
$k \leftarrow arr[i]$	1 assign + 1 index	$2n$
if k in map then	1 map lookup ($O(\log n)$)	$n \log n$
$map[k] \leftarrow map[k] + 1$	1 lookup + 1 add + 1 assign	$3n \log n$
else $map[k] \leftarrow 1$	1 assign	n
Output loop (that stops at 3)	Constant	$O(1)$

$$\begin{aligned} \text{Total Count}(T(n)) &= 1+n+2n+n \log n+3n \log n+n+1 \\ &= 4n \log n+4n+2 \end{aligned}$$

$$\begin{aligned} \checkmark \text{ Skip the constants} &\rightarrow \cancel{4n \log n} + \cancel{4n} + \cancel{2} \\ &= n \log n + n \end{aligned}$$

✓ We have to drop the lower order terms which is n =

$$T(n) = O(n \log n)$$

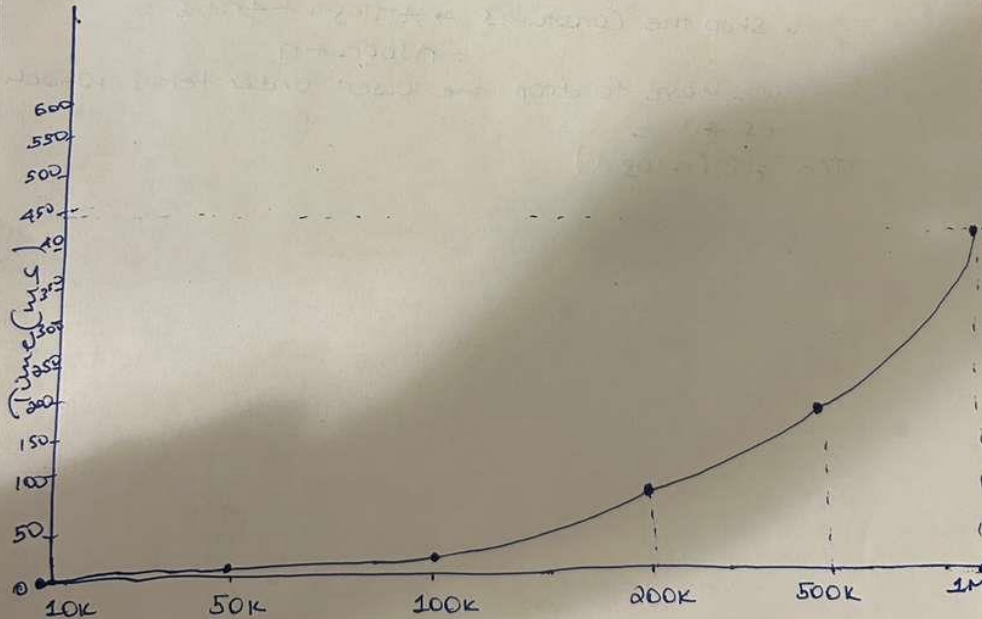
C) Empirical Time Comparison

Size	Algorithm 1 (ms)	Algorithm 2 (ms)	Algorithm 3 (ms)
10000	0	0	5
50000	0	1	14
100000	1	0	26
200000	1	0	82
500000	1	0	202
1000000	1	0	437

→ Algorithm 2 is the fastest it's basically 0ms - one loop minimal operations

→ Algorithm 1 is also very fast but slightly slower than Algorithm 2

→ Algorithm 3 keeps growing much faster - that's the $O(n \log n)$ showing.



Question 2

- 1, 10
- $\log(\log n)$
- $\log n = \ln n$
- $n^{1/k}$ ($k > 3$)
- $n^{1/3}$
- $n^{1/3} \log n$
- $n^{1/2}$
- $n^{1/2} \log n$
- $n \log n = \log n^n$
- n^2
- n^3
- n^k ($k > 3$)
- 2^n
- 3^n
- $n!$
- n^n